名称	原理	优点	缺点	流程
CMS	老年代GC尽可能和用户线程并发执行，降低停顿时间	停顿时间短	空间碎片	
G1	面向堆内「任意组合」的局部回收 (Partial GC)	可指定GC暂停时间、无内存碎片	使用环境受限、内存占用高	
ZGC	几乎整个GC过程都与用户线程并发	在任意堆大小时，都可以将GC停顿控制在10ms之内	对象分配速率不高（堆未分代）	
Epsilon	没有GC功能			

	CMS	G1
启动参数	-XX:UseConcMarkSweepGC -XX:ParallelGCThreads（heap workers数量，不指定时默认与处理器核心数相同） -XX:ConGCThreads（conc workers数量） -XX:CMSInitiatingOccupancyFraction（老年代并发标记阈值）	-XX:useG1GC -XX:ParallelGCThreads（heap workers数量，不指定时默认与处理器核心数相同） -XX:ConGCThreads（parallel workers数量） -XX:InitiatingHeapOccupancyPercent（老年代并发标记阈值） -XX:MaxGCPauseMillis（最长停顿时间，默认不设置，默认200ms）
GC思路	老年代GC尽可能和用户线程并发执行，减少停顿时间	> heap内存划分为大小相等的独立区域（默认1MB，最大32MB），每个区域根据需要分配给eden、survivor、old（内存不再固定连续，而是一系列region的动态集合） > 打破年代局限，面向任意regions组成的集合Mixed GC，优先回收垃圾较多的regions
heap初始化	> 创建heap workers > 初始化新生代 (eden、from、to)，初始化老年代 > 创建conc workers > 启动cmsThread：每隔2s检查一次是否要Major GC（System.gc()、使用率超过阈值等） Major GC「标记-清除」 > 初始标记：heap workers标记GC Roots和新生代直接引用的对象 > 并发标记：conc workers标记被初始标记的对象引用的对象 > 并发预清理：标记被Dirty对象直接或间接引用的对象 > 重新标记：heap workers > 并发清除：清除垃圾对象 > 并发重置：更新heap统计数据，重置cm数据结构	> 创建heap workers > 分配heap内存 > 启动cmThread：获取CGC_lock并wait()，进入_waitSet等待notify通知，收到通知后，执行一次并发标记，再次wait() > 初始标记：parallel workers标记survivor区直接引用的活对象，加入_nextMarkBitMap （此时Young GC刚结束，GC Roots直接引用的活对象已加入_nextMarkBitMap，只需再找出survivor直接引用的活对象即可） > 并发标记：parallel workers标记_nextMarkBitMap中活对象引用的对象（这期间引用变化的对象会被记录到Java线程的SATB缓冲区） > 最终标记：heap workers扫描所有Java线程的本地SATB缓冲区，标记活对象 > 统计：heap workers找出老年代活对象比例小于阈值的regions，加入cSet（下次Mixed GC和年轻代一起回收） > 创建parallel workers：ConGCThreads决定线程数量 > 创建HeapRegion，划分regions
Minor GC	> STW全局停顿 > heap workers标记-复制：扫描GC Roots引用的新生代活对象，copy到survivor or old > 清空eden、from > 交换from、to > 恢复运行	> STW全局停顿 > 选择回收集合cSet（根据指定的停顿时间和预测的回收停顿时间选择） > heap workers标记-复制：扫描GC Roots引用的cSet活对象，copy到survivor or old > 扫描GC Roots直接引用的对象obj：如果属于eden区，标记位图，copy到survivor or old，递归遍历obj属性，复制属于eden区的活对象（old区的只标记位图） > 根据RSet找出old区跨代引用的活对象，递归遍历其属性，找出所有eden区的活对象，copy到survivor or old > 此时cSet中的活对象已完成复制，_nextMarkBitMap中包含GC Roots直接引用的所有对象 > 回收cSet中的regions > 检查老年代使用情况，超过阈值时，CGC_lock->notify()，通知cmThread并发标记 > 恢复运行
Full GC	> 设置前台GC标志位，等待后台GC线程让出执行权 > 标记-清除 or 标记-清除-压缩 > 更新前台GC标志位	> 标记-清除-压缩： > heap workers遍历GC Roots，标记活对象 > 计算活对象的新地址，存入对象头 > 活对象的所有引用指向新地址 > 活对象copy到新地址，清理回收regions
优势	> 老年代并发标记，降低停顿时间	> 无内存碎片：提高分配速度 > 由用户控制GC暂停时间，吞吐量保持在较高水平 > 动态确定回收集合
劣势	> 对处理器资源敏感，占用一部分线程，当处理器核心不足4个时，对用户程序的影响可能很大 > 无法处理浮动垃圾（标记过程结束后产生的对象） > 空间碎片较多，无法找到足够大的连续空间分配给大对象	> 适用环境受限：在多核，内存足够大的机器上才能发挥出优势 > footprint（为GC产生的内存占用）比CMS高：每个region都要有一个卡表，因此G1的记忆集占用较多内存空间 > overload（程序运行时的额外执行负载）比CMS高
选取经验	heap容量小于6G到8G之间时，CMS表现优于G1	Heap容量超过8G时，G1更能发挥其优势
关键技术		> 跨代引用：RSet > 浮动垃圾：SATB缓冲区，记录并发过程中的引用变化 > 标记位图：